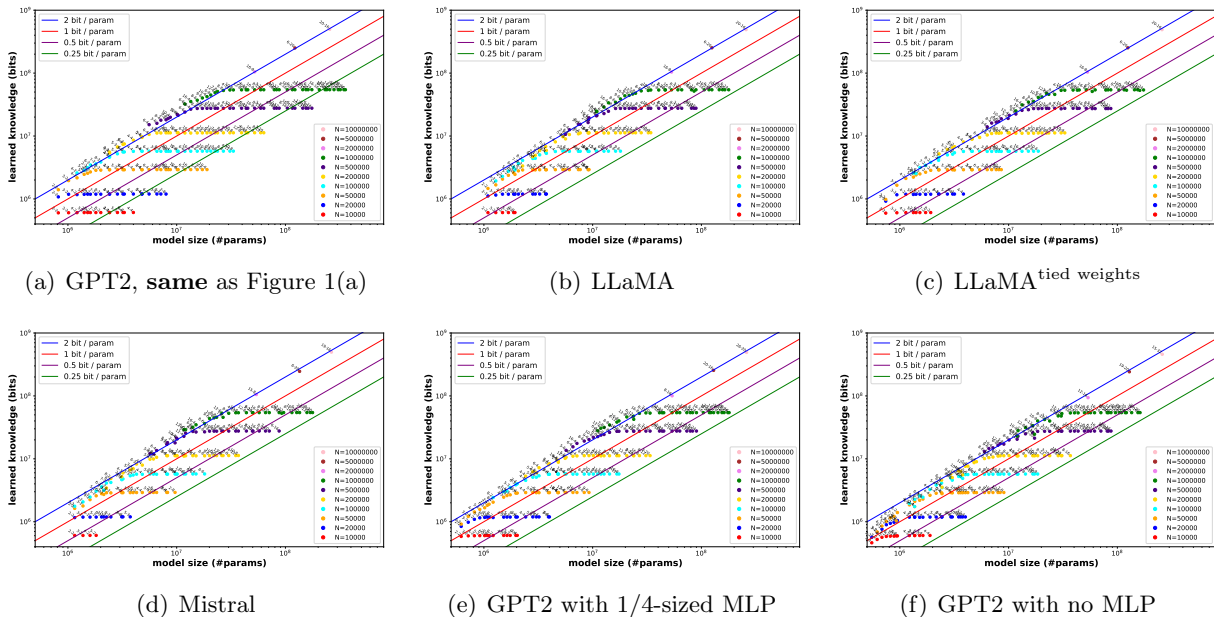




Conclusion. In the 1000-exposure setting, all model architectures closely follow GPT2’s scaling law — including LLaMA/Mistral or even removing the MLP layer completely. The only *very minor* difference is observed in tiny models, where tying the model’s (embedding + output layer) weights enhances its capacity, evident from comparing Figure 3(c) with Figure 3(b).

Result 5 (Figure 3). In the 1000-exposure setting, architectures do not matter much:

- LLaMA architecture performs comparably to GPT2, albeit slightly inferior for the tiny model (i.e., $< 10\text{M}$). This discrepancy can be mitigated by also requiring LLaMA architecture to tie weights, as shown in Figure 3(c) compared to Figure 3(b).
- A similar observation applies to Mistral architecture (see Figure 3(d)).
- Reducing the MLP size of GPT2 architecture by $1/4$ or even eliminating all MLP layers *does not affect* its capacity ratio, see Figure 3(e) and Figure 3(f). This suggests, contrary to conventional beliefs, the Attention layers are also capable of storing knowledge.

This indicates that the 2bit/param capacity ratio is a relatively *universal law* among most typical (decoder-only) language model architectures.

7.1 Insufficient Training Regime and a Closer Comparison

However, differences in architectures become apparent in the insufficient training regime:

Result 6 (Figure 4). In the 100-exposure setting:

- *Even for large models*, LLaMA architecture’s capacity ratio can be 1.3x worse than GPT2, even after optimally tuning learning rates. The results are similar for Mistral.
- Reducing GPT2’s MLP size by 1/4 has a negligible impact on the capacity ratio.
- Removing MLPs decreases the capacity ratio by more than 1.5x.

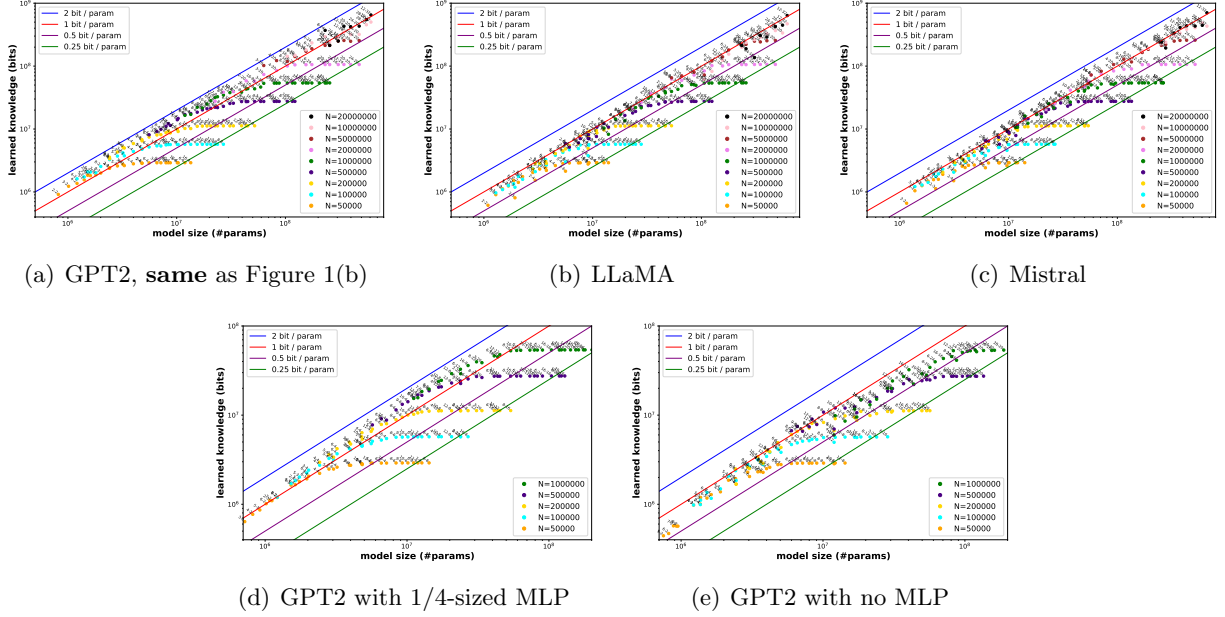


Figure 4: Scaling laws for other model architectures on the bioS(N) data with **100 exposures**.

Conclusion. In the 100-exposure setting, LLaMA/Mistral architectures may underperform GPT2’s scaling law by 1.3x, *even for large models*. Reducing the size of GPT2’s MLP layer by 1/4 does not affect its scaling law, but removing all MLP layers degrades performance. See Figure 5 for a closer comparison.

To investigate *why* the LLaMA architecture is inferior to GPT2 in the 100-exposure (insufficiently trained) setting, we closely examine LLaMA by gradually modifying its architecture *back towards* GPT2 to identify the key architectural changes. We start by tying weights, as this enhances tiny LLaMA model’s capacity in the 1000-exposure setting (Result 5). As illustrated in Figure 5:

- For large models, replacing LLaMA architecture’s gated MLP with a standard MLP (while keeping *silu* unchanged) noticeably improves LLaMA’s capacity ratio.¹⁷
- For tiny LLaMA models, switching back to the GPT2Tokenizer is also necessary to match GPT2’s performance, though this is a minor issue.¹⁸
- Other modifications, such as changing from *silu* to *gelu* or adding trainable biases to layer-norms, do not noticeably affect the capacity ratios (so we ignore those figures).

In summary,

Result 7. In the *insufficient training regime* (notably, the 100-exposure setting), except for tiny models, architectural differences generally do not affect performance, except

- Using gated MLP reduces the model’s capacity ratio (Figure 5);
- Removing all MLP layers lowers the model’s capacity ratio, although significantly reducing the size of MLPs (e.g., by a 1/4 factor) does not.

We propose that our experiments with the controllable biography dataset could serve as a valuable testbed for future architectural designs.

¹⁷As discussed in Appendix B, gated MLP layers are less stable to train, thus requiring more time.

¹⁸This only applies to tiny models and is specific to the biography data we consider here: GPT2Tokenizer may tokenize years such as 1991 into a single token, while LLaMATokenizer will tokenize it into four digit tokens.

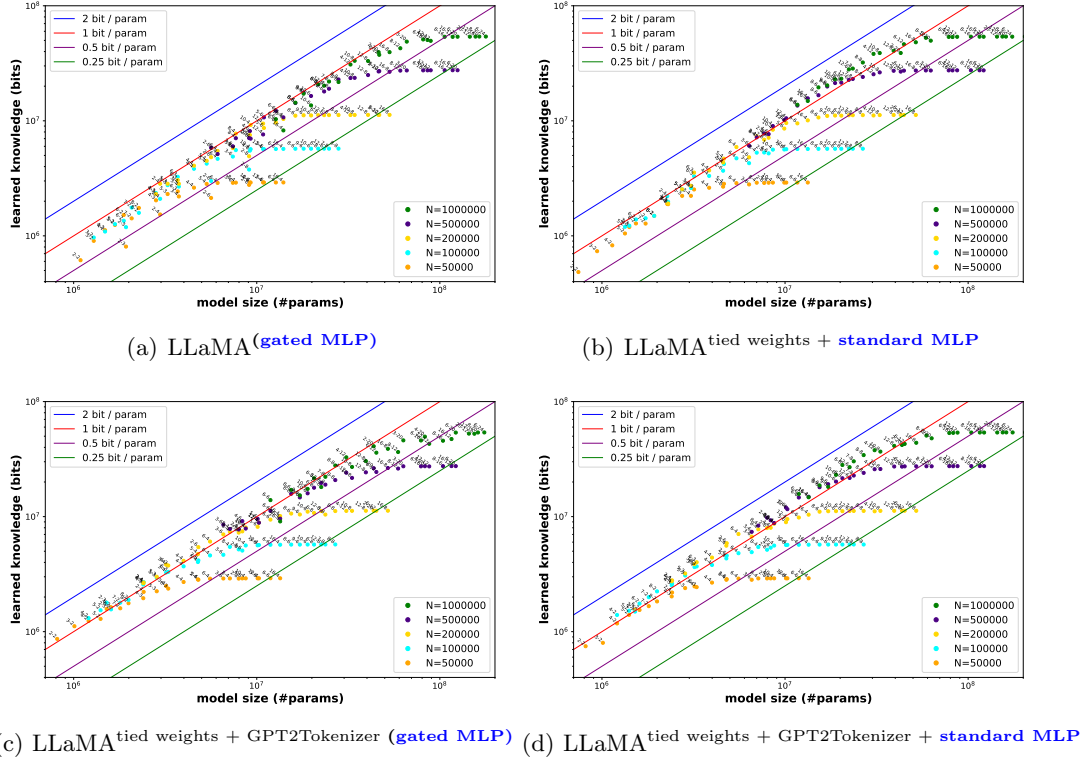


Figure 5: A closer comparison on LLaMA’s scaling laws with bioS(N) data for **100 exposures**.

Conclusion. Switching from gated MLP to a standard MLP (*left vs. right*) enhances larger model’s capacity ratios. For tiny models, using GPT2Tokenizer (*top vs. bottom*) is beneficial (this is a minor point).

Remarks. For a strong comparison we used one learning rate choice in Figure 5(d) and 5(b), but present the best among three choices for Figure 5(a) and 5(c). Further details can be found in Appendix B.2.

8 Quantization vs Scaling Laws

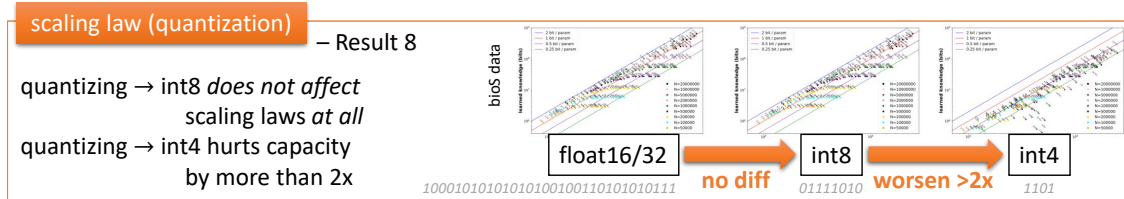


Figure 6: Illustration of Result 8 (see Figure 14 for bioS data and Figure 15 for bioD data in Appendix C).

We have trained and tested models using (mixed precision) 16-bit floats. What happens if we quantize them to int8/int4 after training? We used the `auto_gptq` package, which is inspired by the GPTQ paper [10], for quantization.

Result 8 (Figure 6). Quantizing language models (e.g., GPT2) trained with 16-bit floats:

- to int8 has a negligible impact on their capacity;
- to int4 reduces their capacity by more than 2x.

Thus, **even for** models at peak capacity of 2 bits/param, quantizing to int8 does not affect capacity. Given that 2 bits/param was the best capacity ratio even after 1,000 training exposures on high-quality data, we conclude that extending training *may not* further improve the model’s capacity, *but quantization can*.

Since an int8-based model has an absolute upper bound $R(F) \leq 8$ on capacity ratio, we have:

Corollary 8.1. *Language models, like GPT2, can exceed 1/4 of the absolute theoretical limit for storing knowledge.*

Unfortunately, using this quantization package, reducing the model to int4 significantly diminishes its capacity (more than 2x loss from int8 to int4). This suggests for high-quality int4 models, incorporating quantization during training may be necessary.

8.1 Where Is the Knowledge Stored?

We have seen that LLMs can efficiently compress knowledge into their parameter space, achieving 2bit/param even with 8-bit parameters. This raises the question: how and where is such knowledge stored? Our preliminary answer is that knowledge can be compactly stored within the model in a not-so-redundant manner. It is unlikely that the MLP layers alone store knowledge, as Attention layers, being of comparable sizes, also contribute to knowledge storage (c.f. Result 5). Moreover, particularly in models near the capacity boundary, removing the last transformer layer of an L -layer model to “probe” for remaining knowledge reveals that the “leftover knowledge” can be significantly less than $1 - \frac{1}{L}$ of the total.¹⁹ This suggests knowledge is stored not in individual layers but in a complex manner, akin to a safe with combination locks, where removing one layer may eliminate much more than $\frac{1}{L}$ of the total knowledge.

9 Mixture of Experts vs Scaling Laws

An important way to enhance efficiency in modern language models is the incorporation of sparsity. The Mixture of Experts (MoE) architecture plays a crucial role in this regard [9, 30]. A question arises: does the MoE model scale differently in terms of the capacity ratio? For an MoE model, let P denote the **total number of parameters in the model**, including all experts. Due to its inherent sparsity, the effective number of parameters can be significantly less than P . Our primary observation is that MoE models scale *similarly* to dense models, even with 32 experts per layer.

Consider, for instance, GPT2, but with its MLP layer ($d \rightarrow 4d \rightarrow d$) replaced by 32 experts, each following a $d \rightarrow d \rightarrow d$ configuration. This setup uses $64d^2$ total parameters, but during inference, only $2d^2$ parameters are used per token (e.g., when using $topk = 1$). After including the Attention layers, which each have $4d^2$ parameters, the ratio between the total and the effective number of parameters for the 32-expert MoE models is approximately $\frac{4d^2 + 64d^2}{4d^2 + 2d^2} \approx 11.3$.

One might wonder, given that during inference time, the model uses only 11.3x fewer parameters, whether this affects the model’s capacity ratio by a factor close to 11.3x or closer to 1x? We show:

¹⁹This experiment, deemed not particularly interesting, was omitted from the paper. The probing technique used is Q-probing from [3].